Motor de búsqueda avanzada

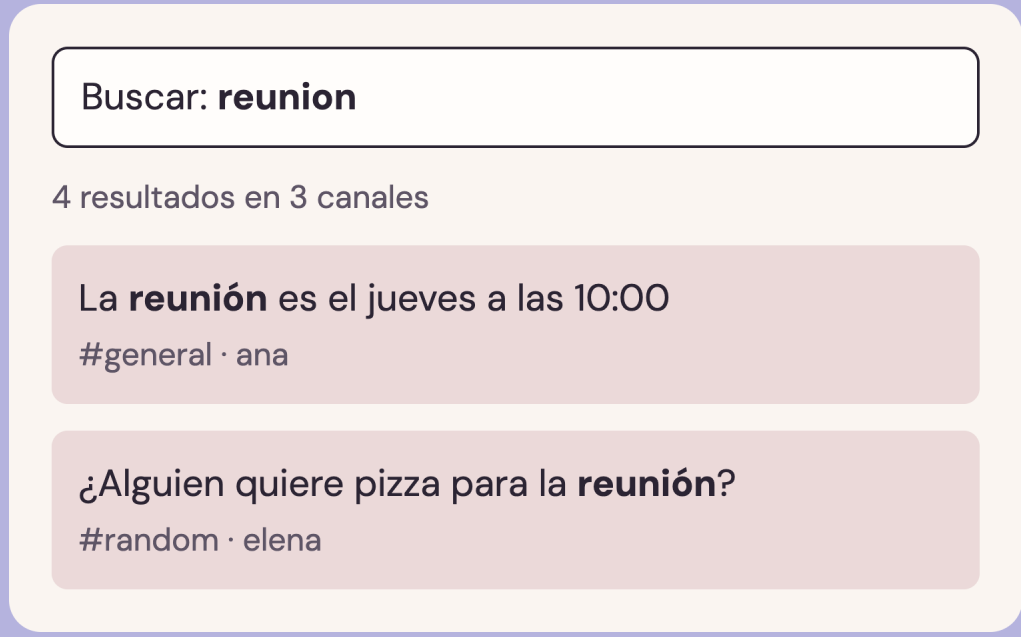
Capacidad vertical para el chat en microservicios

Integrantes
Vicente Carvajal
Sergio Rojas
Fernando Xais

¿Cuál es el problema de usuario?

En un chat, un usuario necesita volver a encontrar un mensaje con un dato específico: un enlace, una fecha o una decisión.

A medida que el chat crece, esos mensajes quedan enterrados y recorrer el historial a mano deja de ser viable.



Contexto de la capacidad

El chat es un contexto efímero: lo escrito ayer queda lejos. La búsqueda funciona como un puente hacia ese historial.

La decisión arquitectónica es la **consistencia eventual**: escribir un mensaje nunca espera a la búsqueda.

Si el indexador o Elasticsearch caen, el chat sigue funcionando y el índice se pone al día cuando vuelven.

0

resultados al buscar justo después de crear el mensaje

1–2 s

después, el mensaje aparece en la búsqueda

Lenguaje ubicuo

El lenguaje de la capacidad se compone de tres términos:

01

Indexación

Proceso asíncrono que convierte cada evento de mensaje en un documento optimizado para búsquedas.

02

Consulta

Texto y filtros (autor, canal, mención, link, fechas) con los que el usuario busca información.

03

Coincidencia

Mensaje indexado que calza con la consulta; se devuelve con las palabras encontradas resaltadas.

Frontera de responsabilidad frente al núcleo

Search es responsable de

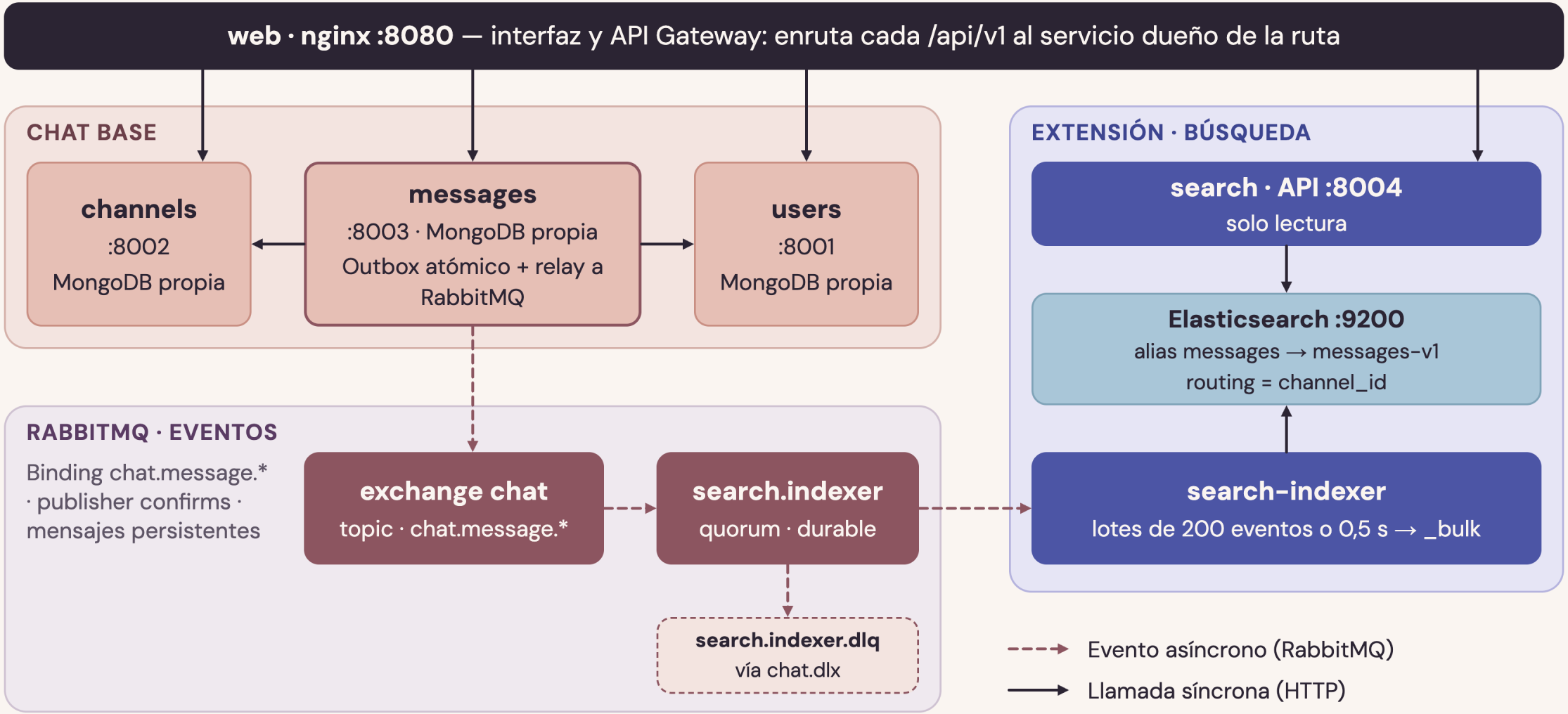
- Indexar los eventos de mensajes creados, editados y borrados
- Resolver consultas y filtros sobre el índice
- Calcular la relevancia y resaltar coincidencias
- Informar el estado del índice: documentos, cola y DLQ

Search nunca

- Modifica ni administra mensajes
- Lee las bases de users, channels o messages
- Llama a sus APIs: todo lo que necesita viaja en el evento
- Publica eventos propios

El índice es una vista proyectada: la fuente de verdad de los mensajes sigue siendo **messages**.

Diagrama de arquitectura



Decisiones de arquitectura

Outbox transaccional

El mensaje y su evento se guardan en una sola escritura atómica; un relay publica con confirmaciones.

Evento con estado

El evento lleva el mensaje completo: search no llama a messages ni lee su base de datos.

Idempotencia

Cada cambio sube la versión; el índice usa versión externa y descarta duplicados o atrasados (409).

Fallos y DLQ

Ack solo cuando Elasticsearch acepta el evento; 429 o 5xx se reencola y los inválidos van a la DLQ.

Indexación por lotes

Hasta 200 eventos o 0,5 s por _bulk; con refresh de 1 s, el mensaje aparece en 1 a 2 s.

Contrato versionado

API en /api/v1, índice messages-v1 tras el alias messages, y errores explícitos: 202 sin índice, 503 sin Elasticsearch.

Listado de APIs, eventos y patrones

Contrato	Rol	Patrón	Contenido
chat.message.created	Consume	Evento asincrónico (RabbitMQ)	Mensaje completo, v1
chat.message.updated	Consume	Evento asincrónico (RabbitMQ)	Edición, versión + 1
chat.message.deleted	Consume	Evento asincrónico (RabbitMQ)	Borrado lógico
GET /api/v1/search/messages	Expone	Request-response sincrónico	Todos los canales
GET /api/v1/channels/{id}/messages/search	Expone	Request-response sincrónico	Un solo canal
GET /api/v1/search/status	Expone	Request-response sincrónico	Indexados, cola y DLQ
GET /api/queues/{cola} (RabbitMQ)	Consume	Request-response sincrónico	Eventos pendientes

Search no publica eventos propios ni llama a las APIs de users, channels o messages: todo lo que necesita viaja en el evento.

Flujo del usuario

1. Usuario A escribe «La reunión es el jueves a las 10:00» en #general. El gateway (nginx) enruta el POST a messages.
2. messages valida canal y autor con GET a channels y users, guarda el mensaje y su evento en una sola escritura atómica (outbox) y responde 201.
3. El relay publica chat.message.created en el exchange chat; la cola search.indexer lo recibe por el binding chat.message.*.
4. search-indexer acumula hasta 200 eventos o 0,5 s y los envía en un _bulk a Elasticsearch; solo después confirma con ack.
5. Usuario B busca «reunion» y el frontend invoca GET /api/v1/search/messages?content=reunion. Search responde con mensaje, autor, canal y fecha, con la palabra resaltada.

Conclusiones

Valor

Cualquier usuario puede recuperar mensajes históricos por texto, autor, canal, mención, link o fecha.

Resiliencia

Si Elasticsearch cae, el chat sigue funcionando; la búsqueda responde 503 y los eventos esperan en la cola.

Consistencia

El outbox evita perder eventos y la versión externa descarta duplicados y eventos fuera de orden.

Limitaciones

Sin backfill de mensajes anteriores a la cola y sin stemming: «reunión» no encuentra «reuniones».

Demo: docker compose up -d --build, python3 seed.py y abrir http://localhost:8080.

¡Muchas gracias!

¿Preguntas?